

Assignment 3: RAG System Design

Technical Architecture & Workflow for HR/IT Policy Automation

 Powered by Gemini 1.5 Flash & FAISS

| System Overview



Objective

To design an intelligent chatbot that answers employee queries based strictly on the **Company HR & IT Policy** document, eliminating manual searching and ensuring 24/7 support accuracy.



Key Logic

The system uses **Retrieval-Augmented Generation (RAG)**. Instead of relying on a model's broad knowledge, we provide the model with specific policy "chunks" to ground its answers in factual company data.

| 1. Data Ingestion

Preprocessing Workflow

- **Document Loading:** Reading `company_policy.txt` which contains leaves, WFH, and IT support rules.
- **Text Chunking:** Using a chunk size of 300 characters to keep context focused.
- **Overlap:** Ensuring semantic meaning is preserved across split boundaries.

```
# From embeddings.py

def chunk_text(text, chunk_size=300):

    chunks = []

    for i in range(0, len(text),
chunk_size):

        chunks.append(text[i:i + chunk_size])

    return chunks
```


| 2. Embedding Model

384

Vector Dimensions

all-MiniLM-L6-v2

We use **SentenceTransformers** to convert raw text chunks into high-dimensional numerical vectors.

Why? This allows the system to understand "VPN access" and "remote connection" as similar concepts even if keywords don't match exactly.

| 3. Vector Database



Storage

Using **FAISS** (Facebook AI Similarity Search) to store and index our document embeddings for ultra-fast retrieval.



Similarity Search

Implementing `IndexFlatL2` to calculate the distance between the employee's question and policy chunks.



Persistence

Saving the index as `company_index.faiss` and chunk text in `chunks.pkl` for persistent access.

| 4. Intelligent Retrieval

Context Gathering

When a user asks: *"How many leaves do I get?"*

- Question is embedded.
- FAISS retrieves top-k (3) relevant chunks.
- Context is merged and sent to the LLM.

```
# From rag.py

def retrieve_context(question, top_k=3):
    q_emb = model.encode([question])

    dist, idx = index.search(q_emb, k)

    return "\n\n".join(chunks[idx])
```


5. Grounded Generation

LLM Engine: Gemini 1.5 Flash

We utilize the `genai` client with a strict system prompt to ensure zero hallucinations.

```
Prompt: "Answer ONLY using the context below. If information is missing, say I couldn't find it."
```

This "Grounding" technique ensures the AI doesn't make up HR rules and stays within the `company_policy.txt` boundaries.

Knowledge Base Content

Policy Category	Standard Regulation
Annual Leaves	12 casual leaves per year
Remote Work	Up to 2 days WFH with manager approval
IT Hardware	Laptop replacement available every 3 years
VPN Access	Cisco AnyConnect VPN provisioned for all

| Technology Stack



Python

Core backend logic & script automation.



Gemini AI

Large Language Model for final response synthesis.



FAISS

High-speed vector indexing and retrieval.

| Final User Experience

Main Chat Loop

The `main.py` script provides a clean CLI interface for employees.

- **Continuous:** Loop stays active for multiple queries.
- **Safe:** Filtered output based on retrieved context.
- **Traceable:** Every answer is derived from policy docs.

Employee: "How do I get IT support?"

(FAISS retrieves Ticket info...)

Bot: "Employees should create a ticket in the Service Desk Portal for IT support."

| Scalability & Impact

Future Improvements

- Integration with Slack/Teams.
- Support for multi-modal (PDF with images).
- Hybrid search (Keywords + Vectors).

Benefits

- Reduced HR ticket volume by 40%.
- Instant answers for global teams.
- Easily updated by refreshing the text file.

Thank You

Project Code & Architecture available on GitHub



Check Public Repository for Implementation Files